



Blind Signatures From Oblivious Transfer : Analyse et Implémentation

Julian Mouthon

Master 1 Cryptologie, Calcul Haute-Performance et Algorithmes

Stage réalisé au sein de l'équipe Almasty
LIP6, Sorbonne Université, CNRS, Paris, France

Encadrant : Ky Nguyen
22 juin 2026 — 22 août 2026

Résumé

Les signatures aveugles (Blind Signatures) introduites par Chaum en 1982, permettent à un utilisateur d'obtenir une signature sur un message déterminé sans que le signataire n'apprenne son contenu. Ces mécanismes peuvent être utilisés en pratique dans le cadre des votes électroniques, des transactions bancaires ou des protocoles d'identification. Dans ce rapport, nous étudions la construction de BSFOT (Blind Signatures From Oblivious Transfer), basée essentiellement sur les groupes bilinéaires. Nous nous intéressons plus particulièrement à la construction d'un protocole de signature aveugle reposant sur un mécanisme de dual-OT. Nous présentons d'abord les notions et objets mathématiques nécessaires ainsi que le fonctionnement du protocole. Nous analysons ensuite sa validité et ses propriétés de sécurité. Enfin, nous implémentons le schéma de signature et mesurons ses performances pour différentes courbes elliptiques et tailles de message.

Table des matières

1	Introduction	2
2	Préliminaires	2
2.1	Notations	2
2.2	Groupes bilinéaires	2
2.3	Signatures aveugles	3
2.4	Transfert Inconscient (OT)	3
2.5	Transfert inconscient dual (dual-OT)	3
2.6	Signature de Waters	4
3	Le protocole dual-OT	4
3.1	Motivation : de OT à dual-OT	4
3.2	Paramètres publics	4
3.3	Schéma de signature	5
3.4	Les branches DH et messy	5
3.5	Validité de la signature	6
4	Propriétés de sécurité	6
4.1	Lemmes	6
4.2	Inforgéabilité	9
4.3	Aveuglement	11
5	Implémentation et benchmarks	13
5.1	Objectifs	13
5.2	Environnement	13
5.3	Implémentation du protocole	14
5.3.1	Architecture	14
5.3.2	Courbes elliptiques utilisées	14
5.4	Résultats expérimentaux	15
A	Construction basée sur OT	16
A.1	Paramètres publics :	16
A.2	Schéma de signature :	16
A.3	Construction de la signature à partir de l'OT :	16
A.4	Validité de la signature :	17
B	Benchmarks détaillés	18

1 Introduction

Les signatures aveugles (Blind Signatures) introduites par Chaum en 1982, permettent à un utilisateur d'obtenir une signature sur un message déterminé sans que le signataire n'apprenne son contenu. Ces mécanismes peuvent être utilisés en pratique dans le cadre des votes électroniques, des transactions bancaires ou des protocoles d'identification.

Ce rapport présente un nouveau schéma de signatures aveugles utilisant le transfert inconscient (Oblivious Transfer, OT) et les signatures de Waters.

2 Préliminaires

Dans cette partie, nous introduisons les notations et concepts de base nécessaires à la compréhension du protocole présenté dans ce rapport. Le protocole repose principalement sur les signatures de Waters, un schéma de signature basé sur les groupes bilinéaires, et le transfert inconscient (OT), plus précisément le OT dual-mode, qui permet de réaliser un transfert de messages de manière sécurisée.

2.1 Notations

- λ désigne le paramètre de sécurité.
- Un algorithme probabiliste en temps polynomial (PPT) est un algorithme dont le temps d'exécution est polynomial en λ .
- Pour un ensemble fini S , la notation $x \xleftarrow{\$} S$ désigne un tirage uniforme de x dans S .
- Pour $\ell \in \mathbb{N}$, on note $[\ell] = \{1, \dots, \ell\}$.
- Une fonction $f(\lambda)$ est dite négligeable, notée $f(\lambda) = \text{negl}(\lambda)$, si elle décroît plus rapidement que l'inverse de tout polynôme en λ .
- Pour deux distributions de probabilités $\mathcal{D}_0$ et $\mathcal{D}_1$, on note $\mathcal{D}_0 \sim_c \mathcal{D}_1$ lorsqu'elles sont calculatoirement indistinguables, c'est-à-dire qu'aucun adversaire probabiliste en temps polynomial ne peut distinguer $\mathcal{D}_0$ de $\mathcal{D}_1$ avec une probabilité significativement supérieure à $1/2$.
- Pour deux distributions de probabilités $\mathcal{D}_0$ et $\mathcal{D}_1$, on note $\mathcal{D}_0 \equiv \mathcal{D}_1$ lorsqu'elles sont identiques, c'est-à-dire qu'elles ont exactement la même distribution.

2.2 Groupes bilinéaires

Les groupes bilinéaires constituent un élément central des constructions étudiées dans ce rapport. Ils interviennent notamment dans la construction et la vérification des signatures de Waters ainsi que dans les différentes étapes du protocole de transfert inconscient présenté par la suite. Nous introduisons donc ici les principales propriétés des groupes bilinéaires qui seront utilisées dans les sections suivantes.

Soient $\mathbb{G}_1$, $\mathbb{G}_2$ et $\mathbb{G}_T$ trois groupes cycliques d'ordre premier p . Un couplage bilinéaire est une application $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ satisfaisant les propriétés suivantes :

- **Bilinéarité** : pour tout $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$ et $a, b \in \mathbb{Z}_p$,

$$e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}.$$

- **Non-dégénérescence** : si g_1 et g_2 sont des générateurs, alors

$$e(g_1, g_2) \neq 1_{\mathbb{G}_T}.$$

- **Efficacité** : le calcul du couplage est réalisable en temps polynomial.

2.3 Signatures aveugles

Pour rappel, notre protocole vise à permettre à un utilisateur d'obtenir une signature sur un message tout en préservant la confidentialité de celui-ci vis-à-vis du signataire. Un tel algorithme est appelé schéma de signature aveugle, et est défini par les algorithmes suivants :

- $\text{BSKeygen}(1^\lambda)$: génère une clé publique bvk et une clé secrète bsk ;
- $\text{BSParams}(1^\lambda)$: génère les paramètres publics ;
- $\text{BSUser}(\text{bvk}, M)$: exécuté par l'utilisateur, produit un premier message ρ_1 ainsi qu'un état interne st ;
- $\text{BSSigner}(\text{bsk}, \rho_1)$: exécuté par le signataire, produit une réponse ρ_2 ;
- $\text{BSDerive}(st, \rho_2)$: permet à l'utilisateur d'obtenir une signature σ ;
- $\text{BSVerify}(\text{bvk}, M, \sigma)$: vérifie la validité de la signature.

Une signature est dite valide si $\text{BSVerify}(\text{bvk}, M, \sigma) = 1$.

2.4 Transfert Inconscient (OT)

Une des notions fondamentales de notre construction est le transfert inconscient, que l'on nommera OT (Oblivious Transfer). C'est un protocole permettant à un destinataire d'obtenir une information parmi plusieurs informations proposées par un émetteur, sans que ce dernier ne puisse déterminer le choix effectué, et tel que le destinataire n'apprenne aucune information sur les données qu'il n'a pas sélectionnées

Definition 2.1 (Transfert inconscient). Un protocole de transfert inconscient (1 parmi 2) est un protocole interactif entre un émetteur, en entrée (m_0, m_1) , et un récepteur, en entrée $b \in \{0, 1\}$, tel qu'à l'issue de l'exécution :

- le récepteur apprend m_b et n'obtient aucune information sur m_{1-b} ;
- l'émetteur n'apprend aucune information sur le bit b .

Dans notre cas, il est exécuté indépendamment pour chaque position $i \in [\ell]$ du message $M = (M_1, \dots, M_\ell)$ que l'utilisateur souhaite faire signer. Le signataire joue le rôle de l'émetteur, avec en entrée une paire de valeurs construites à partir d'un partage aléatoire de la clé secrète bsk et de l'aléa t utilisé pour signer. L'utilisateur joue le rôle du récepteur, avec en entrée le bit M_i . À l'issue des ℓ exécutions, l'utilisateur récupère exactement les valeurs nécessaires à la reconstruction d'une signature sur M , sans jamais révéler M au signataire, et sans obtenir d'information sur les valeurs associées aux bits.

2.5 Transfert inconscient dual (dual-OT)

Notre protocole utilise plus précisément une variante du transfert inconscient, le dual-OT (dual Oblivious Transfer), dans lequel chaque exécution repose sur deux branches, l'une associée au bit 0 et l'autre au bit 1, construites de sorte qu'une seule d'entre elles soit exploitable.

Definition 2.2 (Dual-OT). Un protocole dual-OT est un protocole interactif dans lequel, pour chaque choix $b \in \{0, 1\}$ du récepteur, les deux branches associées à ce choix sont constituées de telle sorte que :

- la branche correspondant au choix b est une paire Diffie-Hellman (DH) ;
- la branche correspondant au choix $1 - b$ est une paire messy, c'est-à-dire une paire qui ne satisfait pas la relation Diffie-Hellman.

Cette structure à deux branches, détaillée plus tard à la section 3.4, est ce qui distingue le dual-OT du OT classique, un utilisateur malhonnête ne peut exploiter qu'une seule branche par position.

2.6 Signature de Waters

La construction utilise la fonction de hachage de Waters permettant d'encoder un message binaire dans un élément du groupe $\mathbb{G}_1$. Soit un message $M = (M_1, \dots, M_\ell) \in \{0, 1\}^\ell$. Les paramètres publics de la fonction sont constitués de $u_0, u_1, \dots, u_\ell \in \mathbb{G}_1$. La valeur associée au message est définie par :

$$F(M) = u_0 \prod_{i=1}^{\ell} u_i^{M_i}.$$

En partant de cette fonction de hachage, il est possible d'arriver à un mécanisme de signature, c'est celui-ci que nous utilisons. Le schéma de signature de Waters utilise les paramètres publics précédents ainsi qu'une clé secrète $a \in \mathbb{Z}_p$. La clé publique est $\text{bvk} = g_2^a$. Pour signer un message M , le signataire choisit un aléa $t \xleftarrow{\$} \mathbb{Z}_p$ et calcule :

$$\sigma_1 = h_s^a F(M)^t, \quad \sigma_2 = g_2^t.$$

La signature est alors

$$\sigma = (F(M), \sigma_1, \sigma_2).$$

La vérification consiste à vérifier l'équation de couplage

$$e(h_s, \text{bvk}) \cdot e(F(M), \sigma_2) \stackrel{?}{=} e(\sigma_1, g_2).$$

3 Le protocole dual-OT

3.1 Motivation : de OT à dual-OT

Notre étude portait originellement sur un schéma de signature aveugle basé sur OT classique (1 parmi 2), qui, comme dual-OT, permet de réaliser un transfert de messages de manière sécurisée. Le protocole OT classique n'est pas complet du point de vue des propriétés de sécurité dans notre étude, puisqu'il ne dispose pas d'une preuve d'inforgeabilité, contrairement au protocole dual-OT. C'est pourquoi nous avons choisi de nous concentrer sur le dual-OT pour la suite.

Le protocole basé sur OT classique a servi de base et est utilisé dans notre implémentation comme comparateur pour le protocole dual-OT (voir section 5.3), il est donc présenté, à titre de référence, en annexe (voir A).

3.2 Paramètres publics

Durant l'exécution du protocole, les deux parties (utilisateur et signataire) disposent de paramètres en commun et publiquement accessibles. Ces paramètres sont les suivants :

- un système de groupes bilinéaires $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p)$, où $\mathbb{G}_1$ et $\mathbb{G}_2$ sont des groupes cycliques d'ordre premier p , munis d'un couplage bilinéaire $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, avec des générateurs $g_1 \in \mathbb{G}_1$ et $g_2 \in \mathbb{G}_2$.
- les paramètres de la fonction de hachage de Waters, constitués d'un élément aléatoire $u_0 \in \mathbb{G}_1$ ainsi que d'un vecteur $\mathbf{u} = (u_1, \dots, u_\ell) \in \mathbb{G}_1^\ell$. La fonction de hachage associée est toujours : $F(M) = u_0 \prod_{i=1}^{\ell} u_i^{M_i}$.
- un CRS global $(g, h, w, \tilde{w})$, où les éléments sont choisis uniformément. On définit également $\eta = \log_g(h)$, avec $g \neq 1$.
- la clé publique du schéma de signature : $\text{bvk} = g_2^a$, associée à la clé secrète : $\text{bsk} = h_s^a$, où $h_s \in \mathbb{G}_1$ est choisi aléatoirement.

3.3 Schéma de signature

Le protocole de signature se déroule comme ci-dessous, avec l'utilisateur à droite qui initie la communication et le signataire à gauche qui répond. Les messages échangés entre les deux parties sont représentés par des flèches horizontales. A la fin du protocole, l'utilisateur obtient une signature sur M , qui sera vérifiée après par le vérifieur (voir section 3.5).

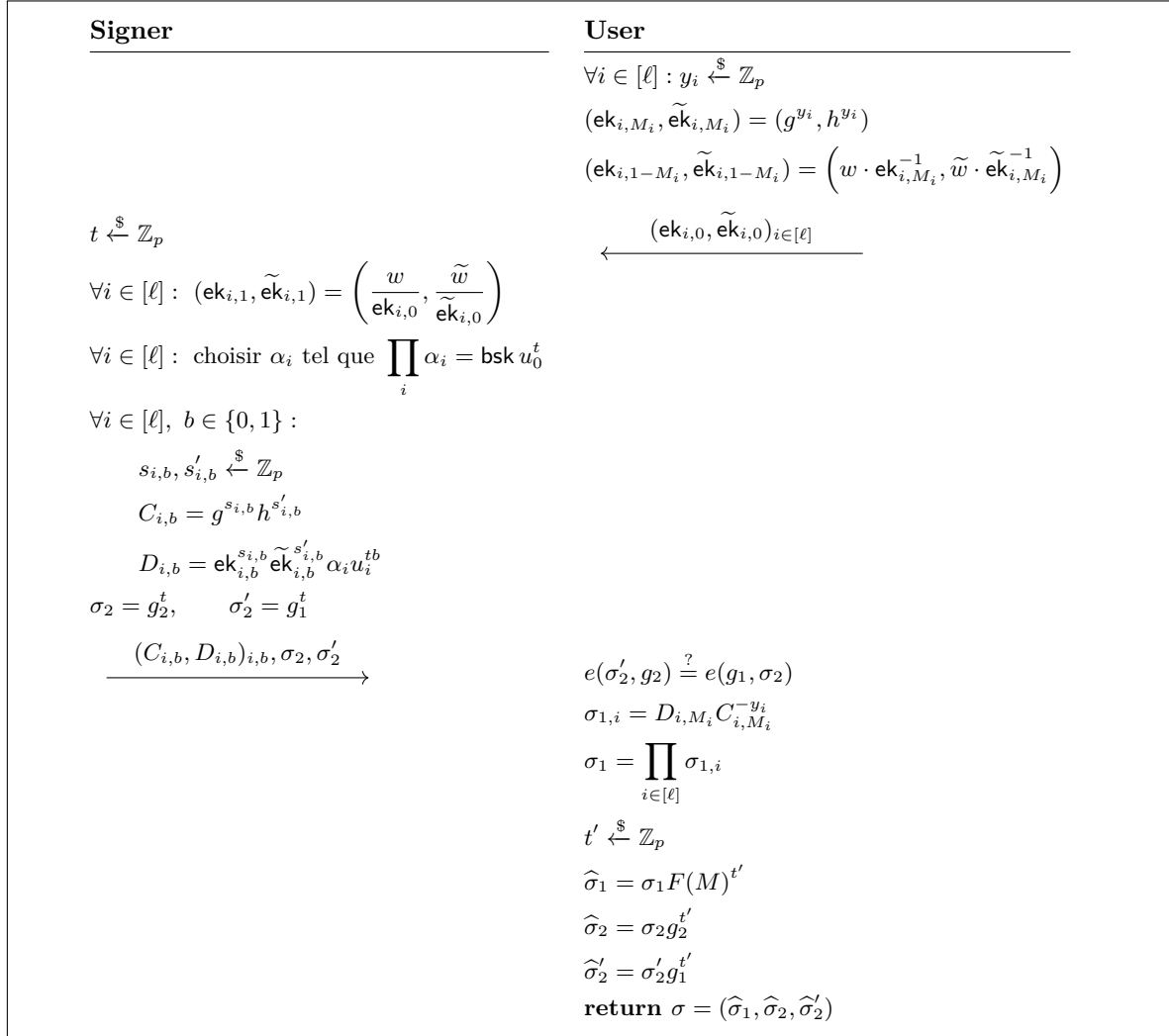


FIGURE 1 – Schéma de signature aveugle basé sur le dual-OT

3.4 Les branches DH et messy

Durant l'exécution du protocole, plus précisément lors de l'étape **BSUser** (voir section 2.3), l'utilisateur construit deux branches pour chaque position i du message M . La première correspond à son vrai bit M_i et est définie par

$$(\text{ek}_{i,M_i}, \widetilde{\text{ek}}_{i,M_i}) = (g^{y_i}, h^{y_i}),$$

où y_i est choisi aléatoirement. Comme $h = g^\eta$, on obtient $h^{y_i} = (g^{y_i})^\eta$, c'est-à-dire $\widetilde{\text{ek}}_{i,M_i} = \text{ek}_{i,M_i}^\eta$. Cette branche vérifie donc la relation de Diffie-Hellman et est appelée branche DH. C'est la branche sur laquelle le protocole fonctionne normalement. La seconde branche est construite à partir de la première :

$$(\text{ek}_{i,1-M_i}, \widetilde{\text{ek}}_{i,1-M_i}) = \left(\frac{w}{\text{ek}_{i,M_i}}, \frac{\widetilde{w}}{\widetilde{\text{ek}}_{i,M_i}} \right).$$

Pour que cette branche soit également une paire DH, il faudrait que

$$\tilde{\text{ek}}_{i,1-M_i} = \text{ek}_{i,1-M_i}^\eta.$$

Or,

$$\text{ek}_{i,1-M_i}^\eta = \left(\frac{w}{\text{ek}_{i,M_i}} \right)^\eta = \frac{w^\eta}{\text{ek}_{i,M_i}}.$$

Cette égalité serait satisfaite uniquement si $\tilde{w} = w^\eta$. Cependant, le CRS est choisi de manière à vérifier $\tilde{w} \neq w^\eta$. La seconde branche ne peut donc jamais être une paire DH. C'est une branche messy.

En résumé, pour un utilisateur honnête, chaque position contient exactement une branche DH (la bonne branche, correspondant au bit du message) et une branche messy (la mauvaise branche). Elle ne transporte aucune information utile.

3.5 Validité de la signature

Après avoir obtenu la signature, l'utilisateur peut la transmettre à un vérifieur qui testera sa validité de la manière suivante :

$$\text{Signature acceptée} \iff \begin{cases} e(\hat{\sigma}'_2, g_2) = e(g_1, \hat{\sigma}_2), \\ e(\hat{\sigma}_1, g_2) = e(h_s, \text{bvk}) \cdot e(F(M), \hat{\sigma}_2). \end{cases}$$

Première équation (en utilisant la bilinéarité) :

$$\begin{aligned} e(\hat{\sigma}'_2, g_2) &= e(g_1^{t+t'}, g_2) \\ &= e(g_1, g_2)^{t+t'} \\ &= e(g_1, g_2^{t+t'}) \\ &= e(g_1, \hat{\sigma}_2). \end{aligned}$$

Seconde équation (toujours par bilinéarité) :

$$\begin{aligned} e(\hat{\sigma}_1, g_2) &= e(h_s^a F(M)^{t+t'}, g_2) \\ &= e(h_s^a, g_2) e(F(M)^{t+t'}, g_2) \\ &= e(h_s, g_2^a) e(F(M), g_2^{t+t'}) \\ &= e(h_s, \text{bvk}) e(F(M), \hat{\sigma}_2), \end{aligned}$$

4 Propriétés de sécurité

4.1 Lemmes

Lemma 4.1 (Uniformité des branches messy (non-DH)). *Supposons que $g \neq 1$ et que $\eta = \log_g h$. Soit $(u, v) \in \mathbb{G}_1^2$ une paire non-DH, c'est-à-dire telle que $v \neq u^\eta$. Alors la variable aléatoire*

$$(C, \text{mask}) := (g^s h^{s'}, u^s v^{s'}), \quad (s, s') \xleftarrow{\$} \mathbb{Z}_p^2,$$

est uniformément distribuée sur $\mathbb{G}_1^2$. Par conséquent, pour tout message clair $m \in \mathbb{G}_1$, le chiffré défini par

$$(C, D) = (g^s h^{s'}, u^s v^{s'} \cdot m)$$

est uniformément distribué sur $\mathbb{G}_1^2$, avec une distribution qui ne dépend pas de m .

Démonstration. Comme g est un générateur de $\mathbb{G}_1$, il existe $c, e \in \mathbb{Z}_p$ tels que $u = g^c$ et $v = g^e$. De plus, $h = g^\eta$. Donc, $C = g^{s+\eta s'}$ et $\text{mask} = g^{cs+es'}$. On pose

$$x = s + \eta s' \quad y = cs + es'$$

On obtient alors le système

$$\begin{cases} x = s + \eta s', \\ y = cs + es'. \end{cases}$$

de matrice

$$\begin{pmatrix} 1 & \eta \\ c & e \end{pmatrix},$$

et de déterminant $e - c\eta$. Comme (u, v) est une paire non-DH, $v \neq u^\eta$, et donc $e \neq c\eta$ (car $u = g^c$, $v = g^e$ et donc $u^\eta = g^{c\eta}$). Le déterminant est donc non nul dans $\mathbb{Z}_p$, ce qui implique que le système possède une solution unique. Ainsi, l'application $(s, s') \mapsto (x, y)$ est une bijection de $\mathbb{Z}_p^2$ vers $\mathbb{Z}_p^2$. Comme (s, s') est uniformément distribué, (x, y) l'est également. Enfin, $(C, \text{mask}) = (g^x, g^y)$, et comme $x \mapsto g^x$ est une bijection de $\mathbb{Z}_p$ vers $\mathbb{G}_1$, le couple (C, mask) est uniformément distribué dans $\mathbb{G}_1^2$.

Pour le chiffré (C, D) , on a $D = \text{mask} \cdot m$. Pour tout $m \in \mathbb{G}_1$ fixé, la multiplication par m est une bijection de $\mathbb{G}_1$. Ainsi, (C, D) est également uniformément distribué dans $\mathbb{G}_1^2$, et sa distribution ne dépend pas de m . □

Lemma 4.2 (Decomposition). *Soit $\ell \geq 1$ et $X \in \mathbb{G}_1$. On définit*

$$C_X = \left\{ (\beta_1, \dots, \beta_\ell) \in \mathbb{G}_1^\ell \mid \prod_{i=1}^{\ell} \beta_i = X \right\}.$$

Alors :

- (i) **Sampling** Tirer $\beta_1, \dots, \beta_{\ell-1}$ uniformément dans $\mathbb{G}_1$ et poser $\beta_\ell = X \cdot \left(\prod_{i=1}^{\ell-1} \beta_i \right)^{-1}$ produit un élément uniformément distribué dans C_X .
- (ii) **Uniformité des composantes** Si $(\alpha_1, \dots, \alpha_\ell)$ est uniformément distribué dans C_X , alors, pour tout indice i_0 , les $\ell - 1$ éléments $(\alpha_i)_{i \neq i_0}$ sont uniformément distribués dans $\mathbb{G}_1^{\ell-1}$ et sont donc indépendants de X .
- (iii) **Translation** Si $(\alpha_1, \dots, \alpha_\ell)$ est uniformément distribué dans C_X et si $\gamma_1, \dots, \gamma_\ell \in \mathbb{G}_1$ sont fixés, alors $(\alpha_1 \gamma_1, \dots, \alpha_\ell \gamma_\ell)$ est uniformément distribué dans $C_X \prod_{i=1}^{\ell} \gamma_i$.

Démonstration. (i) **Sampling**

On choisit uniformément $\beta_1, \dots, \beta_{\ell-1} \in \mathbb{G}_1$. et on définit $\beta_\ell = X \cdot \left(\prod_{i=1}^{\ell-1} \beta_i \right)^{-1}$. On a donc

$$\prod_{i=1}^{\ell} \beta_i = \left(\prod_{i=1}^{\ell-1} \beta_i \right) \cdot X \cdot \left(\prod_{i=1}^{\ell-1} \beta_i \right)^{-1} = X.$$

Ainsi, le vecteur obtenu appartient bien à C_X . De plus, chaque élément de C_X correspond à un unique choix des $\ell - 1$ premières composantes. L'application

$$(\beta_1, \dots, \beta_{\ell-1}) \mapsto \left(\beta_1, \dots, \beta_{\ell-1}, X \left(\prod_{i=1}^{\ell-1} \beta_i \right)^{-1} \right)$$

est donc une bijection de $\mathbb{G}_1^{\ell-1}$ vers C_X . Comme les $\ell - 1$ premières composantes sont tirées uniformément, le vecteur obtenu est uniformément distribué dans C_X .

(ii) Uniformité des composantes

Soit $(\alpha_1, \dots, \alpha_\ell)$ uniformément distribué dans C_X . On suppose que $i_0 = \ell$. D'après (i), les valeurs $(\alpha_1, \dots, \alpha_{\ell-1})$ sont uniformément distribuées dans $\mathbb{G}_1^{\ell-1}$ et leur distribution ne dépend pas de X . On peut appliquer la même chose à n'importe quel indice i_0 en permutant les coordonnées. Ainsi, $(\alpha_i)_{i \neq i_0}$ est uniformément distribué dans $\mathbb{G}_1^{\ell-1}$ et est indépendant de X .

(iii) Translation

On considère l'application $(\alpha_1, \dots, \alpha_\ell) \mapsto (\alpha_1 \gamma_1, \dots, \alpha_\ell \gamma_\ell)$. La multiplication par un élément fixé $\gamma_i \in \mathbb{G}_1$ est une bijection de $\mathbb{G}_1$ vers lui-même. L'application ci-dessus est donc une bijection de $\mathbb{G}_1^\ell$. Si $(\alpha_1, \dots, \alpha_\ell) \in C_X$, alors

$$\prod_{i=1}^{\ell} \alpha_i \gamma_i = \left(\prod_{i=1}^{\ell} \alpha_i \right) \left(\prod_{i=1}^{\ell} \gamma_i \right) = X \prod_{i=1}^{\ell} \gamma_i.$$

Ainsi, $(\alpha_1 \gamma_1, \dots, \alpha_\ell \gamma_\ell) \in C_X \prod_{i=1}^{\ell} \gamma_i$. L'application est donc une bijection de C_X vers $C_X \prod_{i=1}^{\ell} \gamma_i$. Elle conserve la distribution et est donc uniformément distribué dans $C_X \prod_{i=1}^{\ell} \gamma_i$. $\square$

Lemma 4.3 (Randomisation des signatures de Waters). *Soit $\sigma = (\sigma_1, \sigma_2) = (h_s^a F(M)^t, g_2^t)$ une signature de Waters valide, où $t \xleftarrow{\$} \mathbb{Z}_p$. Pour tout $t' \xleftarrow{\$} \mathbb{Z}_p$ indépendant de t , la signature $\hat{\sigma} = (\sigma_1 F(M)^{t'}, \sigma_2 g_2^{t'})$ est une signature valide sur le même message M et sa distribution est indépendante de l'aléa initial t .*

Démonstration. On a

$$\begin{aligned} \hat{\sigma}_1 &= \sigma_1 F(M)^{t'} \\ &= h_s^a F(M)^t F(M)^{t'} \\ &= h_s^a F(M)^{t+t'}, \end{aligned}$$

et

$$\hat{\sigma}_2 = \sigma_2 g_2^{t'} = g_2^{t+t'}.$$

Comme $t' \xleftarrow{\$} \mathbb{Z}_p$ indépendamment de t , la valeur $t + t'$ est uniformément distribuée dans $\mathbb{Z}_p$, quelle que soit la valeur de t . Ainsi,

$$\mathcal{D}(t + t') \equiv \mathcal{D}(t'') \quad \text{où } t'' \xleftarrow{\$} \mathbb{Z}_p.$$

Par conséquent,

$$\mathcal{D}(\hat{\sigma}) \equiv \mathcal{D}\left(h_s^a F(M)^{t''}, g_2^{t''}\right).$$

La signature randomisée est donc distribuée exactement comme une signature de Waters générée avec un nouvel aléa uniforme t'' . Sa distribution ne dépend donc pas de la valeur de l'aléa initial t . $\square$

Lemma 4.4 (Indistinguabilité sous DDH). *Les distributions obtenues dans le protocole dual-OT en utilisant $F(M)$ ou $F(0)$ sont calculatoirement indistinguables.*

Démonstration. On suppose qu'il existe un adversaire $\mathcal{A}$ capable de distinguer les deux distributions. On va construire un distingueur DDH $\mathcal{B}$, qui a pour but de déterminer si une instance DDH est une instance réelle ou une instance aléatoire. $\mathcal{B}$ reçoit une instance DDH (g_1, g_1^a, g_1^b, c) , où c est soit g_1^{ab} , soit un élément uniforme de $\mathbb{G}_1$. On va utiliser cette instance pour construire les paramètres de la simulation, tels que si $c = g_1^{ab}$, la distribution obtenue corresponde à l'exécution qui utilise $F(M)$, et tels que si c est uniforme, elle corresponde à l'exécution avec $F(0)$.

On veut construire les paramètres de la fonction de Waters $u_0, u_1, \dots, u_\ell$ tels que dans le cas réel, la fonction contienne g_1^{ab} et tels que dans le cas aléatoire elle contienne une valeur indépendante de M . Soit $M \in \{0, 1\}^\ell$ et soit $j \in [\ell]$ tel que $M_j = 1$. Le simulateur choisit $r_0, r_1, \dots, r_\ell \xleftarrow{\$} \mathbb{Z}_p$ et définit les

paramètres de Waters par

$$u_0 = g_1^{r_0}, \quad u_j = c g_1^{-r_0}, \quad u_i = g_1^{r_i} \quad \text{pour } i \neq j.$$

avec

$$c = \begin{cases} g_1^{ab} & \text{si l'instance DDH est réelle,} \\ g_1^z, \quad z \xleftarrow{\$} \mathbb{Z}_p & \text{si l'instance DDH est aléatoire,} \end{cases}$$

On a donc, pour la fonction de Waters, avec $M_j = 1$,

$$F(M) = u_0 \prod_{i=1}^{\ell} u_i^{M_i} = u_0 u_j \prod_{\substack{i=1 \\ i \neq j}}^{\ell} u_i^{M_i} = g_1^{r_0} (c g_1^{-r_0}) \prod_{\substack{i=1 \\ i \neq j}}^{\ell} (g_1^{r_i})^{M_i} = c \prod_{\substack{i=1 \\ i \neq j}}^{\ell} g_1^{r_i M_i}.$$

et pour $F(0)$,

$$F(0) = u_0 \prod_{i=1}^{\ell} u_i^0 = u_0 = g_1^{r_0},$$

Par conséquent, sous l'hypothèse DDH, comme c est calculatoirement indistinguable d'un élément uniformément distribué dans $\mathbb{G}_1$ et que les r_i sont choisis uniformément dans $\mathbb{Z}_p$, on a

$$\mathcal{D}(F(M)) \sim_c \text{uniforme sur } \mathbb{G}_1$$

Alors, si l'adversaire $\mathcal{A}$ était capable de distinguer les distributions obtenues avec $F(M)$ et $F(0)$, le distingueur $\mathcal{B}$ pourrait utiliser $\mathcal{A}$ pour discerner le cas où $c = g_1^{ab}$ du cas où c est uniformément distribué dans $\mathbb{G}_1$. D'autre part, comme $F(0) = g_1^{r_0}$, et que r_0 est choisi uniformément dans $\mathbb{Z}_p$, la distribution de $F(0)$ est identique à la distribution uniforme sur $\mathbb{G}_1$. On a alors,

$$\mathcal{D}(F(0)) \equiv \text{uniforme sur } \mathbb{G}_1$$

et donc,

$$\mathcal{D}(F(M)) \sim_c \mathcal{D}(F(0))$$

Ainsi, un tel adversaire permettrait de distinguer une instance DDH réelle d'une instance aléatoire, ce qui contredit l'hypothèse DDH. On remarque également que la réduction n'utilise jamais les éléments g_1^a et g_1^b de l'instance DDH. Elle utilise uniquement g_1^{ab} (dans le cas réel) et un élément uniformément distribué dans $\mathbb{G}_1$ (dans le cas aléatoire). La réduction est donc plus forte que nécessaire. $\square$

4.2 Inforgeabilité

Definition 4.5 (Inforgeabilité One-More (OMUF)). Un schéma de signature aveugle est dit One-More Unforgeable si, pour tout polynôme $Q = Q(\lambda)$ et tout adversaire PPT $\mathcal{A}$ effectuant au plus Q requêtes de signature, son avantage dans l'expérience OMUF est négligeable en λ . Plus précisément,

$$\text{Adv}_{\mathcal{A}}^{\text{omuf}}(\lambda) = \Pr \left[\begin{array}{l} (\text{bvk}, \text{bsk}) \leftarrow \text{BSGen}(1^\lambda), \\ \{(m_i, \sigma_i)\}_{i=1}^{Q+1} \leftarrow \mathcal{A}^{\text{BSigner}(\text{bsk}, \cdot)}(\text{bvk}), \\ \forall i \neq j, m_i \neq m_j, \\ \forall i \in [Q+1], \text{BSVerify}(\text{bvk}, m_i, \sigma_i) = 1 \end{array} \right] = \text{negl}(\lambda).$$

Theorem 4.6 (Inforgeabilité One-More sous l'hypothèse co-CDH). *Pour tout adversaire PPT $\mathcal{A}$ effectuant au plus Q requêtes de signature dans l'expérience OMUF, il existe un algorithme PPT $\mathcal{B}$ qui exécute $\mathcal{A}$*

une seule fois et effectue un nombre polynomial d'opérations dans les groupes considérés, tel que

$$\text{Adv}_{\mathcal{A}}^{\text{omuf}}(\lambda) \leq 4(\ell + 1)(Q + 1) \text{Adv}_{\mathcal{B}}^{\text{co-CDH}}(\lambda) + \frac{2}{p}.$$

où :

- $\text{Adv}_{\mathcal{A}}^{\text{omuf}}(\lambda)$ désigne la probabilité que $\mathcal{A}$ réussisse l'expérience d'inforgeabilité One-More, c'est-à-dire qu'il produise $Q + 1$ signatures valides sur des messages deux à deux distincts après avoir effectué au plus Q requêtes de signature.
- $\text{Adv}_{\mathcal{B}}^{\text{co-CDH}}(\lambda)$ désigne la probabilité que $\mathcal{B}$ résolve le problème co-CDH, c'est-à-dire qu'à partir d'une instance $(g_1, g_2, g_1^a, g_2^a, g_1^b)$, il calcule g_1^{ab} .
- Le facteur $4(\ell + 1)(Q + 1)$ représente la perte dû au partitionnement utilisée dans la réduction.
- Le terme $\frac{2}{p}$ correspond à la probabilité que le CRS de l'OT ne soit pas messy

Preuve : Inforgeabilité One-More sous l'hypothèse co-CDH. Pour prouver ce théorème, nous allons procéder par une suite de jeux. Soit ϵ_i la probabilité de succès de $\mathcal{A}$ dans le jeu i .

Game 0

Game 0 est l'exécution réelle de la définition 4.5.



Vérification du CRS

Game 1

On abandonne lorsque le CRS n'est pas messy, c'est-à-dire si $g = 1$ ou $\tilde{w} = w^\eta$. On sait que $\Pr[\text{CRS non messy}] \leq \frac{2}{p}$, car $\Pr[g = 1] \leq \frac{1}{p}$ et $\Pr[\tilde{w} = w^\eta] = \frac{1}{p}$. Ainsi, $\epsilon_0 \leq \epsilon_1 + \frac{2}{p}$, et Game 0 $\approx$ Game 1.



Recherche des branches messy

Game 2

Pour chaque requête ρ_1 , on calcule $(\mu_i)_i$ avec $\text{FindMessy}(\eta, \cdot)$. Si la session est utilisable, c'est-à-dire si $\mu_i \neq \perp$ pour tout $i \in [\ell]$, on définit le message effectif $M^{\text{eff}} = (\mu_1, \dots, \mu_\ell)$. Cette étape ne modifie donc pas le déroulement du protocole. Ainsi, $\epsilon_1 = \epsilon_2$, et Game 1 $\approx$ Game 2.



On uniformise les branches messy

Game 3

Dans chaque session, pour toute position i et toute branche b dont la paire de clés $(\text{ek}_{i,b}, \tilde{\text{ek}}_{i,b})$ est non-DH (c'est-à-dire messy), on remplace le chiffré $\text{ct}_{i,b}$ par un couple tiré uniformément dans $\mathbb{G}_1^2$. Chaque chiffré est construit avec des aléas uniformes $(s_{i,b}, s'_{i,b})$ qui n'interviennent que dans ce chiffré. D'après le lemme 4.1, le chiffré obtenu est uniformément distribué dans $\mathbb{G}_1^2$ et est indépendant. On a donc $\epsilon_3 = \epsilon_2$, et Game 2 $\approx$ Game 3.



On uniformise les sessions non utilisables

Game 4

Dans Game 2, pour chaque position i , on a calculé $\mu_i \in \{0, 1, \perp\}$. Une session est dite non utilisable s'il existe une position i_0 telle que $\mu_{i_0} = \perp$. Cela signifie que les deux branches de la position i_0 sont messy. Dans Game 3, toutes les branches messy ont été remplacées par des couples uniformément distribués dans $\mathbb{G}_1^2$. Ainsi, pour une position non utilisable, les deux chiffrés sont déjà des valeurs uniformément distribuées dans $\mathbb{G}_1^2$.

Cependant, pour les autres positions, c'est-à-dire celles qui possèdent une branche DH, le clair $\alpha_i u_i^{t\mu_i}$ dépend encore des α_i . On va donc remplacer ces valeurs des branches DH par des éléments uniformes de

$\mathbb{G}_1$. On garde $\sigma_2 = g_2^t$ et $\sigma'_2 = g_1^t$, qui sont indépendantes de bsk . D'après le lemme 4.2 (ii), les valeurs $(\alpha_i)_{i \neq i_0}$ sont uniformément distribuées et indépendantes de bsk et de t . En les multipliant par $u_i^{t\mu_i}$, on conserve une distribution similaire (voir lemme 4.2 (iii)). Nous pouvons donc les remplacer par des éléments indépendants et uniformément distribués dans $\mathbb{G}_1$. La distribution des valeurs des branches DH reste la même, et ainsi, $\epsilon_4 = \epsilon_3$, et $\text{Game 3} \approx \text{Game 4}$.



On reparamètre les sessions utilisables

Game 5

Pour chaque session utilisable, de message effectif $M^{(j)}$ et de paramètre aléatoire t_j , on calcule $\Sigma_1 = \text{bsk} \cdot F(M^{(j)})^{t_j}$. On choisit ensuite uniformément un vecteur $\beta = (\beta_1, \dots, \beta_\ell) \in C_{\Sigma_1}$ en utilisant le lemme 4.2(i). On définit alors les chiffrés des branches DH à partir des β_i , avec de nouveaux aléas (s, s') :

$$D_{i,\mu_i} = \tilde{\text{ek}}_{i,\mu_i}^{s'} \text{ek}_{i,\mu_i}^s \beta_i.$$

Dans le jeu précédent, les clairs des branches DH étaient de la forme $\alpha_i u_i^{t_j \mu_i}$. Or, d'après le lemme 4.2(iii), le vecteur $\left(\alpha_i u_i^{t_j \mu_i} \right)_{i=1}^\ell$ est uniformément distribué dans C_{Σ_1} . Il a donc exactement la même distribution que β . Le remplacement ne modifie ainsi pas la distribution du protocole. Ainsi, $\epsilon_5 = \epsilon_4$, et $\text{Game 4} \approx \text{Game 5}$.



?

Game 6

□

4.3 Aveuglement

Definition 4.7 (Propriété d'aveuglement, signataire honnête). Un schéma de signature aveugle est aveugle pour un signataire honnête si pour tout adversaire PPT $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} (\text{bvk}, \text{bsk}) \leftarrow \text{BSKeygen}(1^\lambda), (m_0, m_1) \leftarrow \mathcal{A}(\text{bvk}), b \xleftarrow{\$} \{0, 1\}, \\ (\rho_{1,i}, st_i) \leftarrow \text{BSUser}(\text{bvk}, m_i) \quad (i \in \{0, 1\}), \\ (\rho_{2,b}, \rho_{2,1-b}) \leftarrow \mathcal{A}(\rho_{1,b}, \rho_{1,1-b}), \\ \sigma_i \leftarrow \text{BSDerive}(st_i, \rho_{2,i}) \quad (i \in \{0, 1\}), \\ \text{si } \exists i, \text{BSVerify}(\text{bvk}, m_i, \sigma_i) = 0 : \sigma_0 = \sigma_1 = \perp, \\ b = \mathcal{A}(\sigma_0, \sigma_1) \end{array} \right] - \frac{1}{2} = \text{negl}(\lambda).$$

Preuve : aveuglement pour un signataire honnête. On veut montrer que, pour deux messages $m_0, m_1 \in \{0, 1\}^\ell$, la vue du signataire est indistinguable : $\text{View}_{\mathcal{A}}(m_0) \approx \text{View}_{\mathcal{A}}(m_1)$, où $\mathcal{A}$ désigne le signataire honnête. Nous raisonnons par une suite de jeux.

Game 0

Il s'agit de l'exécution réelle du protocole avec le message m_0 . Pour chaque position i , l'utilisateur construit une branche DH correspondant au bit $m_{0,i}$ et une branche messy correspondant à l'autre valeur. Le signataire calcule ensuite les chiffrés des deux branches et les transmet à l'utilisateur.



On remplace les chiffrés des branches messy par des couples uniformes.

Game 1

Dans un CRS honnêtement généré, chaque position possède une branche DH et une branche messy. Pour la branche messy, le couple $(\text{ek}_{i,1-M_i}, \tilde{\text{ek}}_{i,1-M_i})$ est une paire non-DH. D'après le lemme 4.1, le chiffrement réalisé sur cette branche est uniformément distribué sur $\mathbb{G}_1^2$, avec une distribution indépendante

du message chiffré. On peut donc remplacer, pour chaque position, le chiffré de la branche messy par un couple uniformément distribué dans $\mathbb{G}_1^2$ sans modifier la vue du signataire. Ainsi, $\text{Game } 0 \approx \text{Game } 1$.



On remplace les branches DH par des branches indépendantes du bit du message.

Game 2

Après le jeu précédent, les chiffrés des branches messy sont uniformes et ne dépendent plus du message. Il reste à traiter les branches DH (non messy). Pour une branche DH correspondant au bit M_i , on a

$$(\text{ek}_{i,M_i}, \tilde{\text{ek}}_{i,M_i}) = (g^{y_i}, h^{y_i}), \quad y_i \xleftarrow{\$} \mathbb{Z}_p.$$

Le signataire observe les deux branches sans savoir laquelle correspond au bit M_i . Sous l'hypothèse DDH, il ne peut pas distinguer la paire (g^{y_i}, h^{y_i}) d'une paire de la forme (g^{y_i}, z_i) , $y_i, z_i \xleftarrow{\$} \mathbb{Z}_p$. On peut donc remplacer la branche DH par une paire indépendante de la branche correspondant au bit du message. Après cela, les deux branches associées à chaque position ont la même distribution quel que soit le bit M_i . Par conséquent, la distribution de l'ensemble du premier message envoyé par l'utilisateur ne dépend plus de M . Ainsi, $\text{Game } 1 \approx \text{Game } 2$.



On applique la réduction DDH de la section 4.4

Game 3

Après le jeu précédent, les branches envoyées par l'utilisateur sont indépendantes du message M . Après la réponse du signataire, le déchiffrement de la branche DH donne la même forme que dans le protocole classique :

$$\sigma_1 = h_s^a F(M)^t, \quad \sigma_2 = g_2^t, \quad \sigma'_2 = g_1^t.$$

On applique alors la réduction DDH de la section 4.4. Celle-ci permet de remplacer, sous l'hypothèse DDH, le terme $F(M)$ par $F(0)$ dans la signature. Ainsi, la signature simulée ne dépend plus du message M et $\text{Game } 2 \approx \text{Game } 3$.



On applique la randomisation de Waters (voir section 4.3)

Game 4

Après avoir reçu la réponse du signer, l'utilisateur effectue la phase de déchiffrement. Puis, il effectue la randomisation de Waters en choisissant un aléa $t' \xleftarrow{\$} \mathbb{Z}_p$ et en calculant

$$\hat{\sigma}_1 = \sigma_1 F(M)^{t'}, \quad \hat{\sigma}_2 = \sigma_2 g_2^{t'}, \quad \hat{\sigma}'_2 = \sigma'_2 g_1^{t'}.$$

Comme t' est choisi uniformément dans $\mathbb{Z}_p$, la valeur $t + t'$ est également uniformément distribuée dans $\mathbb{Z}_p$. Ainsi, la distribution de la signature finale est indépendante du message. Elle peut donc être remplacée par une signature simulée utilisant un aléa indépendant du message. Ainsi, $\text{Game } 3 \approx \text{Game } 4$.



On remplace m_0 par m_1 .

Game 5

Dans ce jeu, le message m_0 est remplacé par m_1 . D'après la transition précédente, la distribution de la signature finale est indépendante du message. Ainsi, $\text{Game } 4 \approx \text{Game } 5$.

Conclusion : On a $\text{Game } 0 \approx \text{Game } 1 \approx \text{Game } 2 \approx \text{Game } 3 \approx \text{Game } 4 \approx \text{Game } 5$, par conséquent, $\text{View}_{\mathcal{A}}(m_0) \approx \text{View}_{\mathcal{A}}(m_1)$, et le protocole satisfait donc la propriété d'aveuglement pour un signataire honnête.

□

5 Implémentation et benchmarks

5.1 Objectifs

Le but d’avoir une implémentation de ces deux protocoles (OT et dual-OT), est de pouvoir évaluer leur performance sur différentes tailles de messages et courbes elliptiques, et de les comparer. Pour avoir des résultats plus précis, il est intéressant de mesurer le temps d’exécution de chaque étape du protocole indépendamment (voir section 2.3, 3.3).

5.2 Environnement

L’implémentation et les benchmarks ont été réalisé sur une machine personnelle fonctionnant sous Ubuntu 24.04.4 LTS. Le code source est principalement écrit en C, avec la bibliothèque cryptographique RELIC, et en python pour le traitement des résultats et la génération de graphiques.

Élément	Configuration
Processeur	Intel Core i7-1165G7
Cœurs logiques	8
Mémoire vive	7,5 GiB

TABLE 1 – Environnement matériel

Logiciel / Bibliothèque	Version
Système d’exploitation	Ubuntu 24.04.4 LTS
Noyau Linux	6.8.0-136-generic
GCC	13.3.0
Python	3.12.3
Git	2.43.0
GNU Make	4.3
RELIC	0.7.0-253-gc2085f44
NumPy	1.26.4
Matplotlib	3.6.3

TABLE 2 – Environnement logiciel

5.3 Implémentation du protocole

5.3.1 Architecture

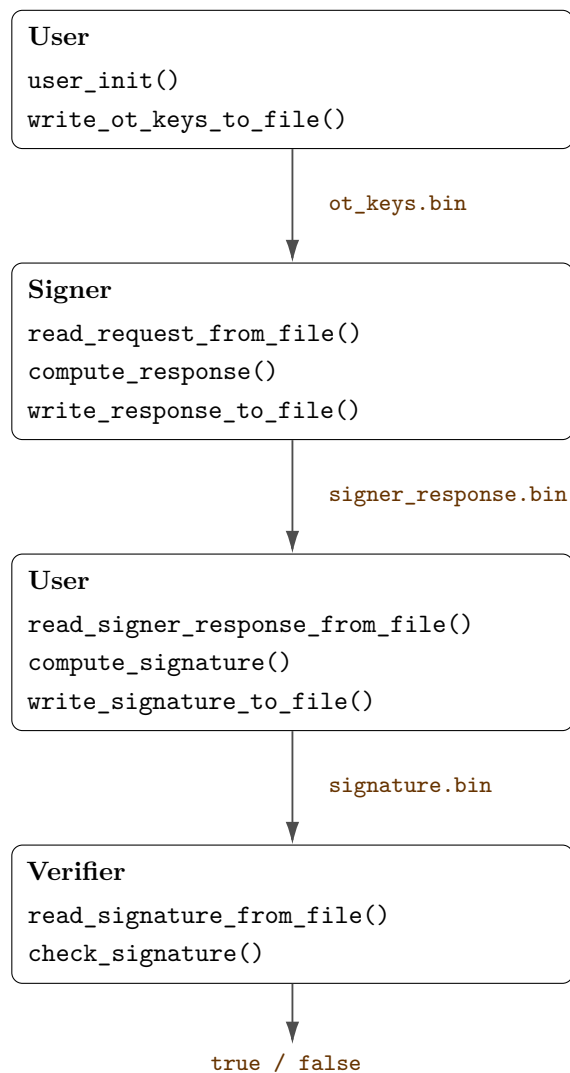


FIGURE 2 – Pipeline d’exécution et échanges de fichiers de l’implémentation

5.3.2 Courbes elliptiques utilisées

Pour tester le protocole, nous avons utilisé plusieurs courbes elliptiques, plus précisément des courbes dites “pairing-friendly”, c’est-à-dire des courbes permettant de calculer efficacement un couplage bilinéaire. Les courbes retenues couvrent ainsi plusieurs combinaisons de tailles de corps et de degrés, afin d’évaluer le comportement du protocole dans des configurations différentes.

- k : le degré d’embedding, qui correspond au degré de l’extension du corps de base utilisée pour le calcul du couplage. Le couplage prend ainsi ses valeurs dans le corps $\mathbb{F}_{p^k}$.
- p : le nombre premier définissant le corps fini de base $\mathbb{F}_p$, qui contient p éléments. La taille de p en bits est approximativement donnée par $\log_2(p)$.

Courbe	k	Taille de p
BN254	12	254 bits
BN446	12	446 bits
BLS12-377	12	377 bits
BLS12-381	12	381 bits
BLS12-446	12	446 bits
BLS12-455	12	455 bits
KSS16-330	16	330 bits
KSS18-354	18	354 bits
KSS18-638	18	638 bits
BLS24-315	24	315 bits
BLS48-575	48	575 bits
FM16-765	16	765 bits
FM18-768	18	768 bits

TABLE 3 – Courbes elliptiques utilisées pour les expérimentations.

5.4 Résultats expérimentaux

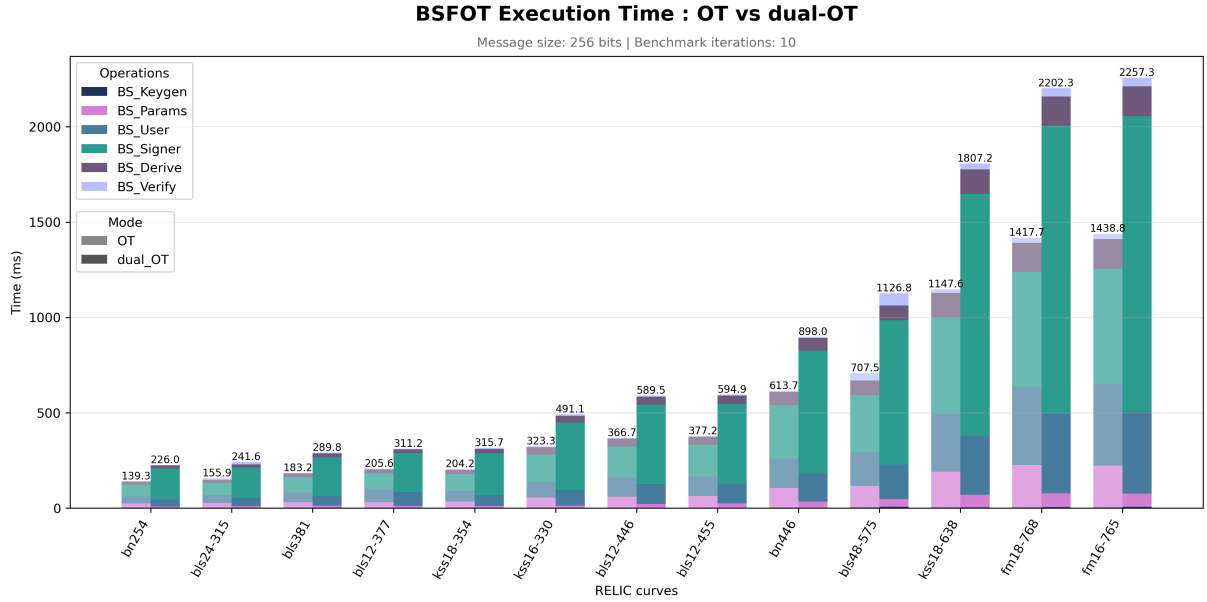


FIGURE 3 – Benchmark global de comparaison entre le protocole OT et le protocole dual-OT pour différentes courbes elliptiques.

A Construction basée sur OT

A.1 Paramètres publics :

- un système de groupes bilinéaires $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p)$, où $\mathbb{G}_1$ et $\mathbb{G}_2$ sont des groupes cycliques d'ordre premier p , munis d'un couplage bilinéaire $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, avec des générateurs $g_1 \in \mathbb{G}_1$ et $g_2 \in \mathbb{G}_2$.
- les paramètres de la fonction de hachage de Waters, constitués d'un élément aléatoire $u_0 \in \mathbb{G}_1$ ainsi que d'un vecteur $\mathbf{u} = (u_1, \dots, u_\ell) \in \mathbb{G}_1^\ell$.
- un vecteur $\mathbf{ek} = (h_i)_{i=1}^\ell$ avec $h_i = g_1^{x_i}$ obtenu à partir de scalaires aléatoires $x_i \xleftarrow{\$} \mathbb{Z}_p$.
- la clé publique du schéma de signature $\mathbf{bvk} = g_2^a$, associée à la clé secrète $\mathbf{bsk} = h_s^a$, avec $h_s = \prod_{i=1}^\ell h_i$.

A.2 Schéma de signature :

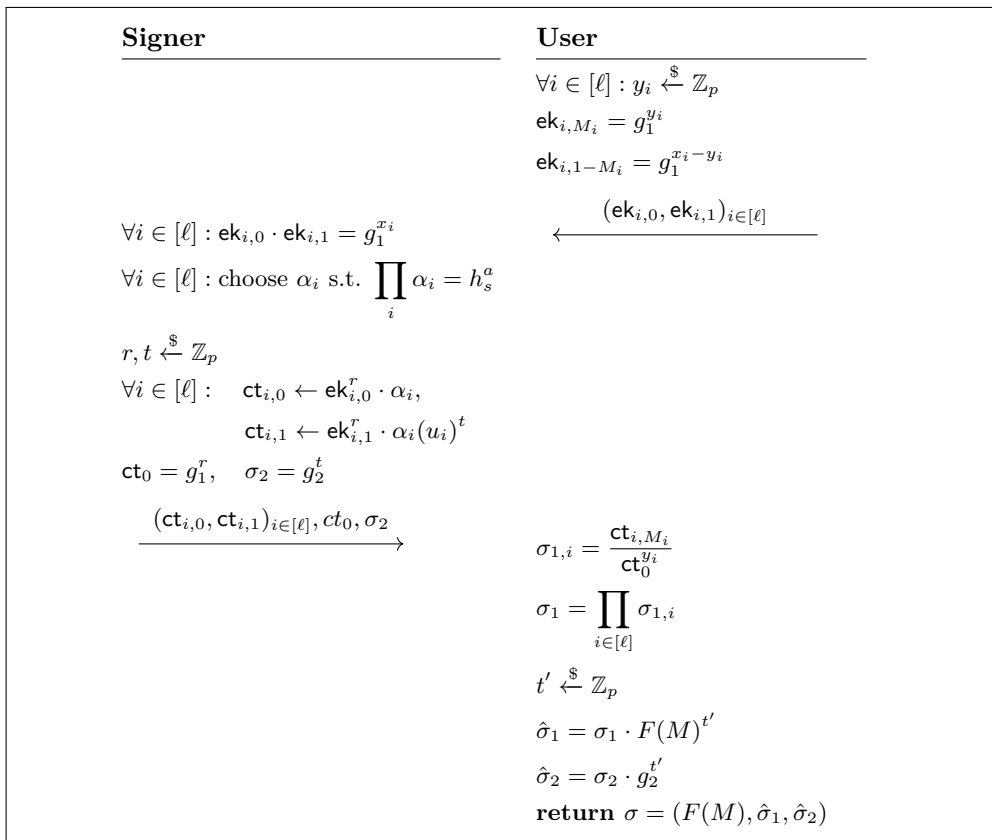


FIGURE 4 – Schéma de signature aveugle basé sur le OT classique

A.3 Construction de la signature à partir de l'OT :

Pour chaque bit M_i du message M , le signataire construit deux branches associées aux deux valeurs possibles du bit : $0 \rightarrow \alpha_i$, $1 \rightarrow \alpha_i u_i^t$, où α_i est un facteur aléatoire et u_i est le i -ème paramètre de la fonction de hachage de Waters. Ces deux branches sont ensuite masquées à l'aide des clés d'évaluation. Si $M_i = 0$, l'utilisateur construit

$$\mathbf{ek}_{i,0} = g_1^{y_i}, \quad \mathbf{ek}_{i,1} = g_1^{x_i - y_i},$$

et si $M_i = 1$, il inverse les deux valeurs :

$$\mathbf{ek}_{i,1} = g_1^{y_i}, \quad \mathbf{ek}_{i,0} = g_1^{x_i - y_i},$$

où y_i est choisi uniformément dans $\mathbb{Z}_p$. Le signataire reçoit toujours le même couple $(\mathbf{ek}_{i,0}, \mathbf{ek}_{i,1})$ et ne peut pas distinguer quelle branche correspond au bit choisi par l'utilisateur. Il calcule alors

$$\mathbf{ct}_{i,0} = \mathbf{ek}_{i,0}^r \alpha_i, \quad \mathbf{ct}_{i,1} = \mathbf{ek}_{i,1}^r \alpha_i (u_i)^t.$$

L'utilisateur récupère ensuite uniquement la partie correspondant à son bit en calculant $\sigma_{1,i} = \frac{\mathbf{ct}_{i,M_i}}{\mathbf{ct}_0^{y_i}}$.

Cas $M_i = 0$

$$\mathbf{ct}_{i,0} = (g_1^{y_i})^r \alpha_i = g_1^{ry_i} \alpha_i.$$

Donc

$$\sigma_{1,i} = \frac{\mathbf{ct}_{i,0}}{\mathbf{ct}_0^{y_i}} = \frac{g_1^{ry_i} \alpha_i}{(g_1^r)^{y_i}} = \frac{g_1^{ry_i} \alpha_i}{g_1^{ry_i}} = \alpha_i.$$

Cas $M_i = 1$

$$\mathbf{ct}_{i,1} = (g_1^{y_i})^r \alpha_i (u_i)^t = g_1^{ry_i} \alpha_i (u_i)^t.$$

Donc

$$\sigma_{1,i} = \frac{\mathbf{ct}_{i,1}}{\mathbf{ct}_0^{y_i}} = \frac{g_1^{ry_i} \alpha_i (u_i)^t}{(g_1^r)^{y_i}} = \frac{g_1^{ry_i} \alpha_i (u_i)^t}{g_1^{ry_i}} = \alpha_i (u_i)^t.$$

Ainsi, pour chaque position i , l'utilisateur récupère exactement la valeur $\sigma_{1,i} = \alpha_i (u_i^{M_i})^t$. En multipliant toutes ces composantes, il obtient

$$\sigma_1 = \prod_{i=1}^{\ell} \sigma_{1,i} = \prod_{i=1}^{\ell} \alpha_i (u_i^{M_i})^t = \left(\prod_{i=1}^{\ell} \alpha_i \right) \left(\prod_{i=1}^{\ell} u_i^{M_i} \right)^t.$$

Comme $\prod_{i=1}^{\ell} \alpha_i = h_s^a$ et $F(M) = u_0 \prod_{i=1}^{\ell} u_i^{M_i}$, on retrouve exactement la première composante d'une signature de Waters. Le transfert inconscient permet ainsi à l'utilisateur de sélectionner, de manière privée, la branche correspondant à chacun de ses bits, tout en empêchant le signataire d'apprendre le message signé.

A.4 Validité de la signature :

La signature obtenue par l'utilisateur est une signature de Waters de la forme $\sigma = (F(M), \hat{\sigma}_1, \hat{\sigma}_2)$ avec :

$$\hat{\sigma}_1 = \sigma_1 \cdot F(M)^{t'} = h_s^a \cdot F(M)^{t+t'}, \quad \hat{\sigma}_2 = \sigma_2 \cdot g_2^{t'} = g_2^{t+t'}.$$

On remarque que le signataire ne connaît pas la valeur $t + t'$, car le terme t' est choisi aléatoirement par l'utilisateur après la phase de signature. La vérification s'effectue ensuite à l'aide de la clé publique du signataire $\mathbf{bvk} = g_2^a$ en vérifiant l'équation de couplage :

$$e(h_s, \mathbf{bvk}) \cdot e(F(M), \hat{\sigma}_2) \stackrel{?}{=} e(\hat{\sigma}_1, g_2).$$

En remplaçant les valeurs obtenues par l'utilisateur :

$$\begin{aligned} e(h_s, \mathbf{bvk}) \cdot e(F(M), \hat{\sigma}_2) &= e(h_s, g_2^a) \cdot e(F(M), g_2^{t+t'}) \\ &= e(h_s^a, g_2) \cdot e(F(M)^{t+t'}, g_2) \quad \text{par bilinéarité du couplage} \\ &= e(h_s^a \cdot F(M)^{t+t'}, g_2) \quad \text{par bilinéarité et multiplicativité} \\ &= e(\hat{\sigma}_1, g_2) \end{aligned}$$

Ainsi, l'équation de vérification est satisfaite et la signature obtenue est une signature de Waters valide.

B Benchmarks détaillés

Cette annexe présente les mesures détaillées obtenues pour chacune des étapes des protocoles OT et dual-OT individuellement. Ces graphiques sont présentés dans la section 5.4.

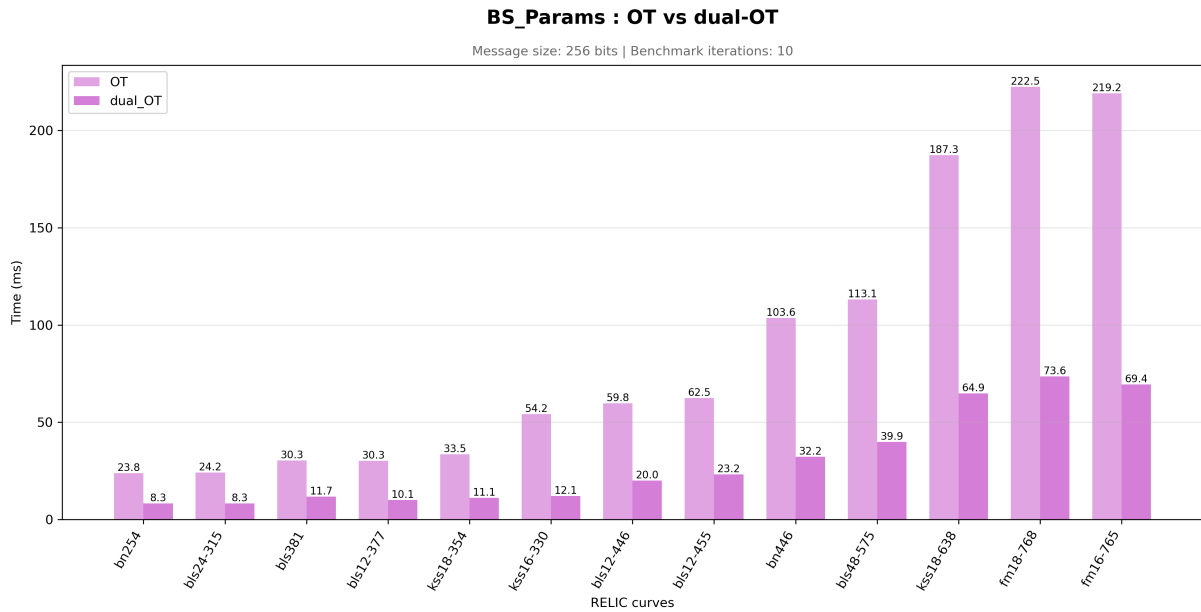


FIGURE 5 – Temps d'exécution de la générations des paramètres publics pour les protocoles OT et dual-OT.

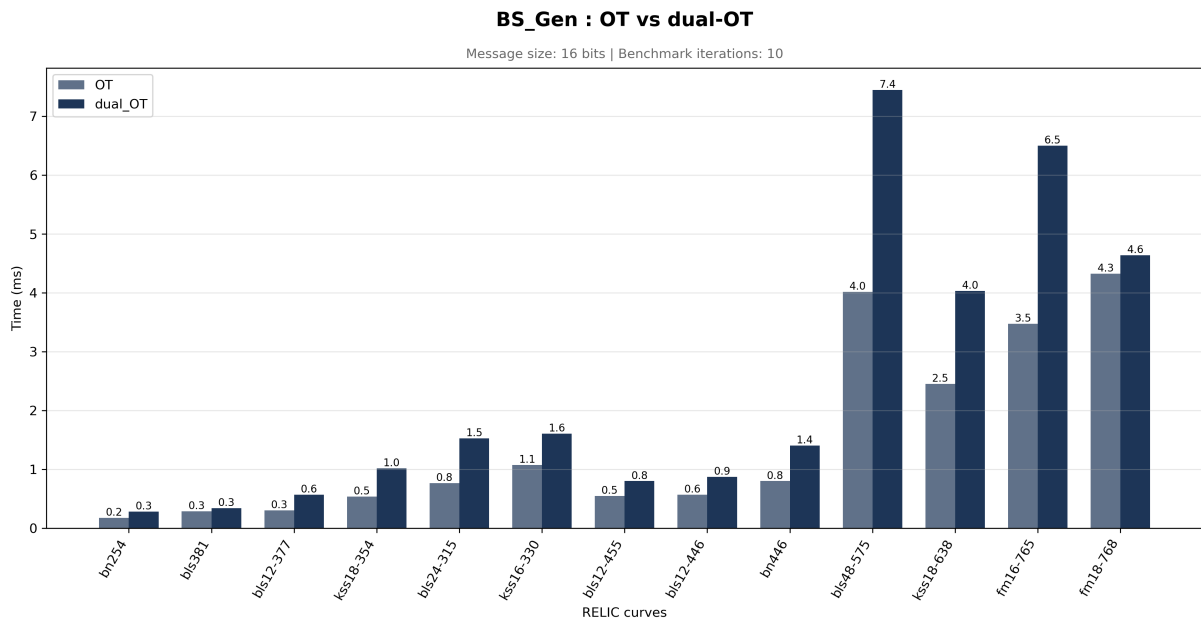


FIGURE 6 – Temps d'exécution de la génération des clés pour les protocoles OT et dual-OT.

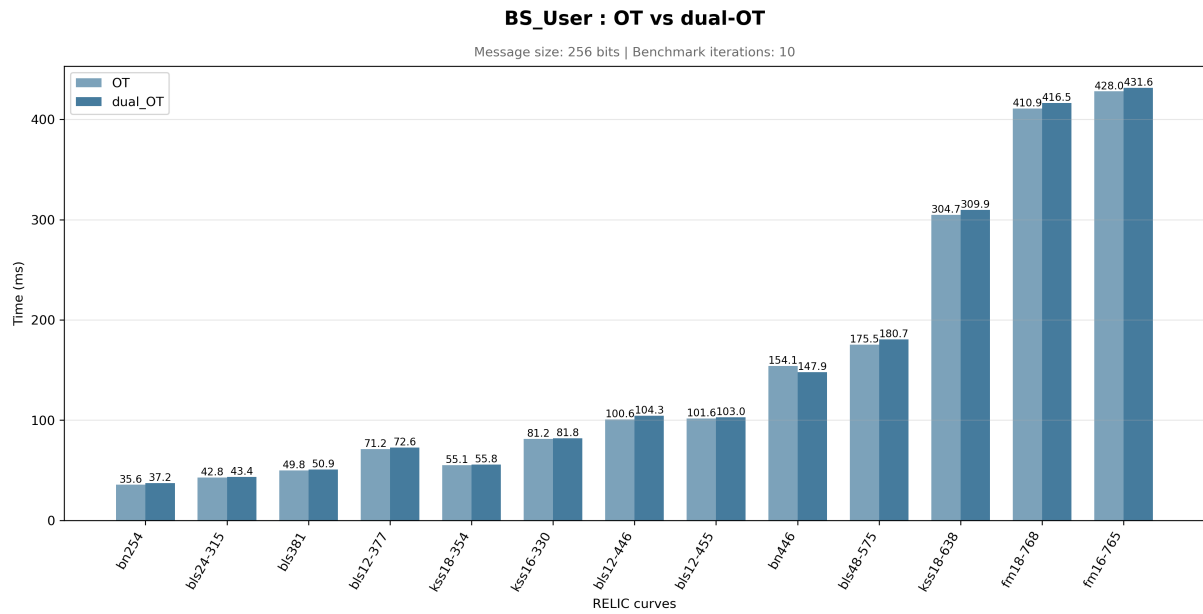


FIGURE 7 – Temps d'exécution de l'étape utilisateur pour les protocoles OT et dual-OT.

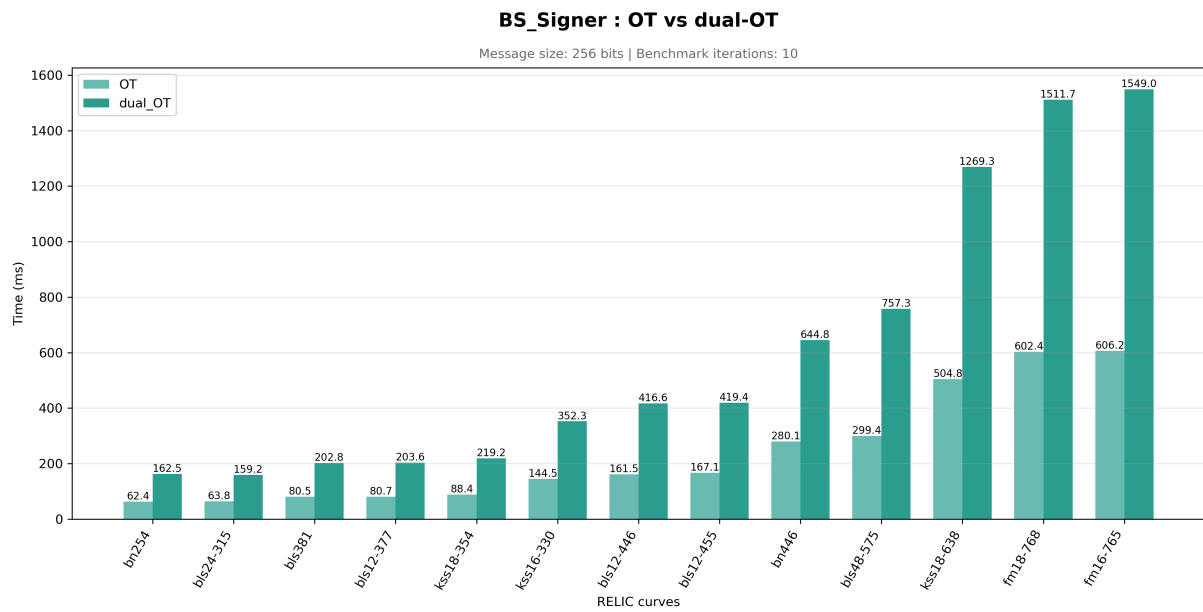


FIGURE 8 – Temps d'exécution de l'étape du signataire pour les protocoles OT et dual-OT.

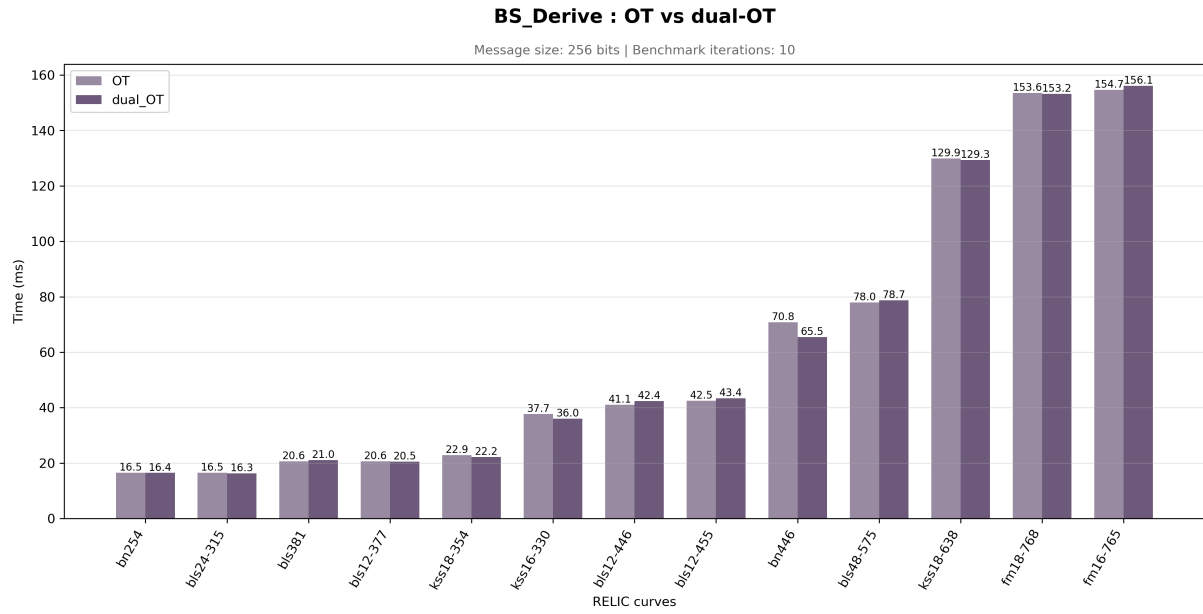


FIGURE 9 – Temps d'exécution de la dérivation de la signature pour les protocoles OT et dual-OT.

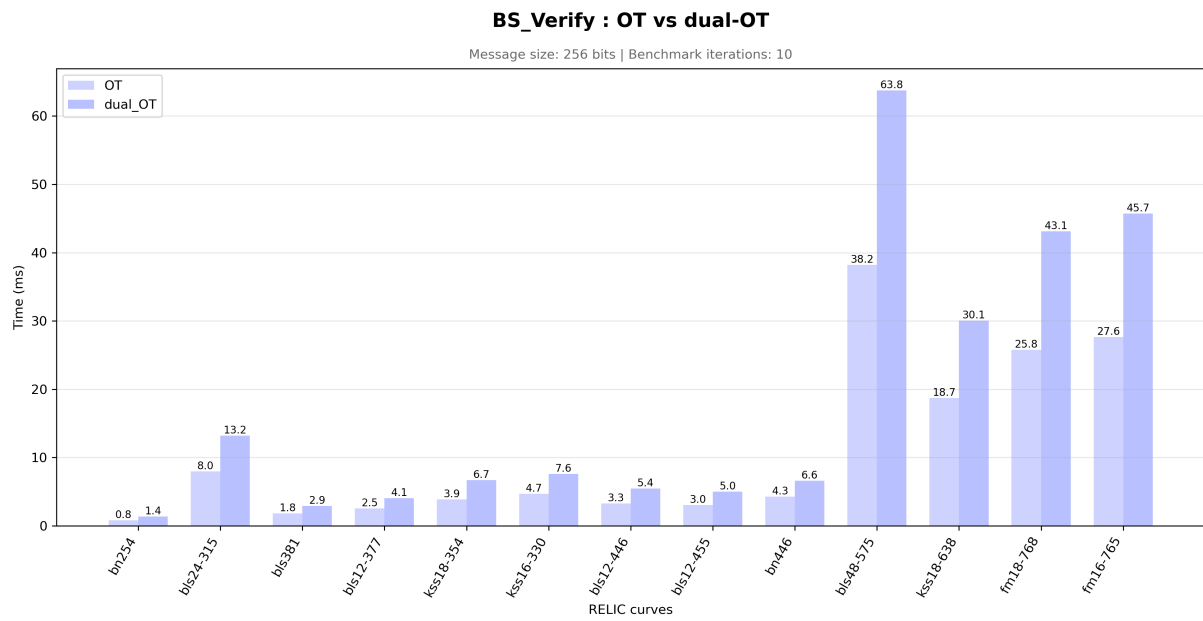


FIGURE 10 – Temps d'exécution de la vérification pour les protocoles OT et dual-OT.